

Esercizi

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2024, 2026
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>

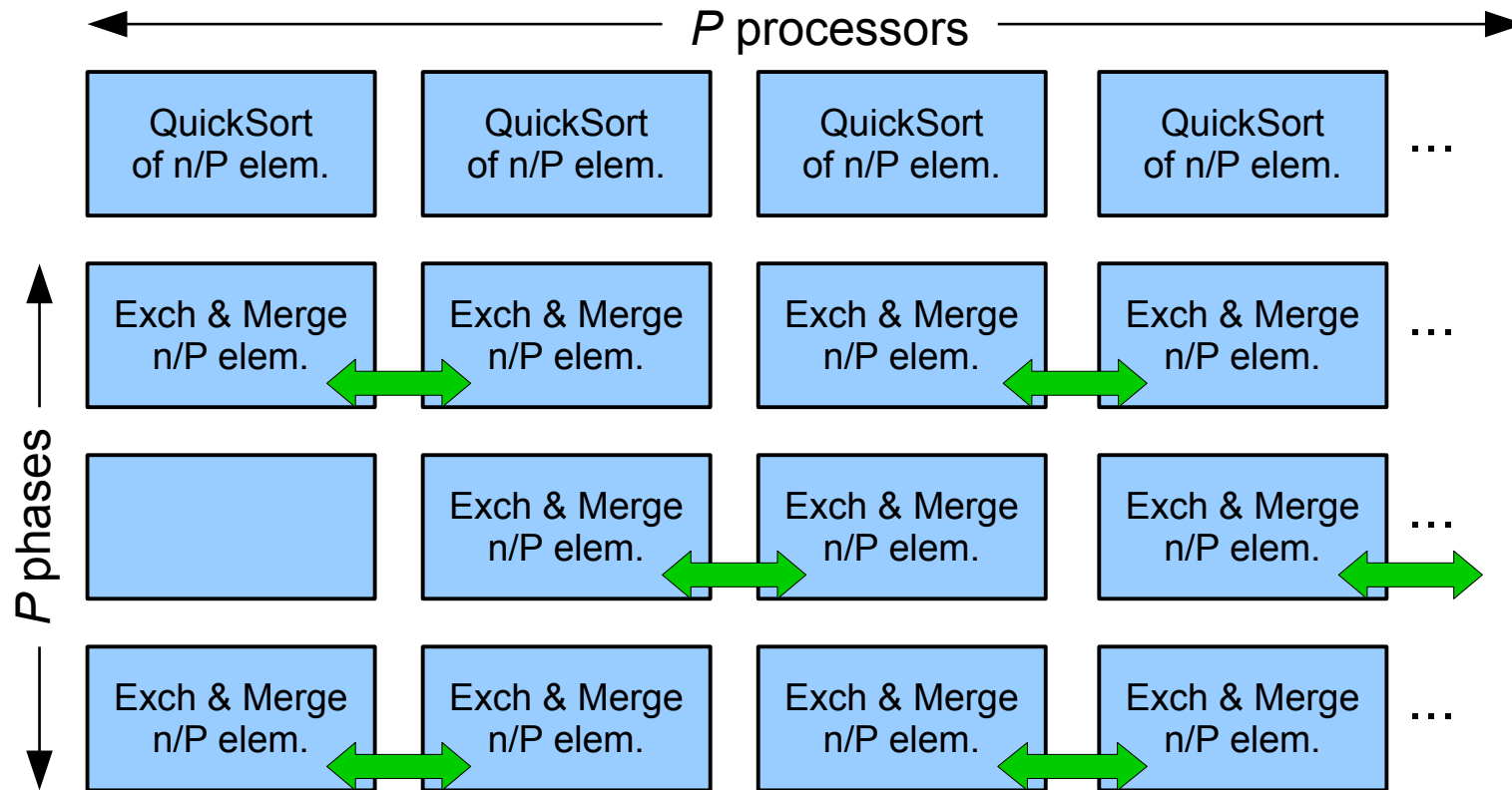


Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Domanda 1

- Compute the **work** and **span** of the distributed-memory odd-even transposition sort under simplifying assumptions:
 - Do not need to scatter the input array initially;
 - Do not need to gather the sorted array at the end;
 - Assume that point-to-point send/receive of k bytes requires $O(k)$ time.

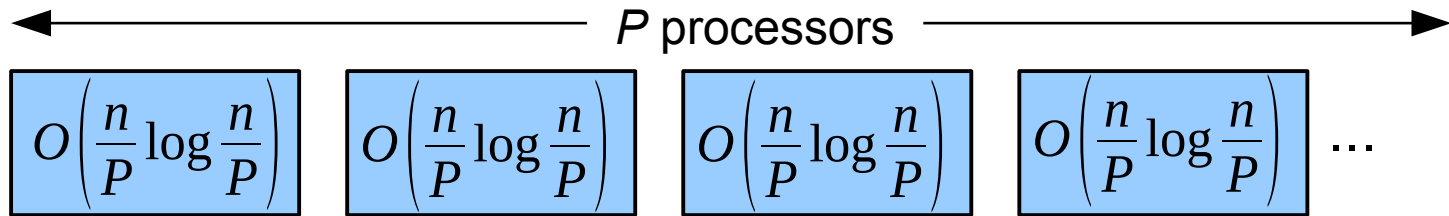
Distributed-memory Odd-Even Sort with P processors



Distributed-memory Odd-Even Sort with P processors

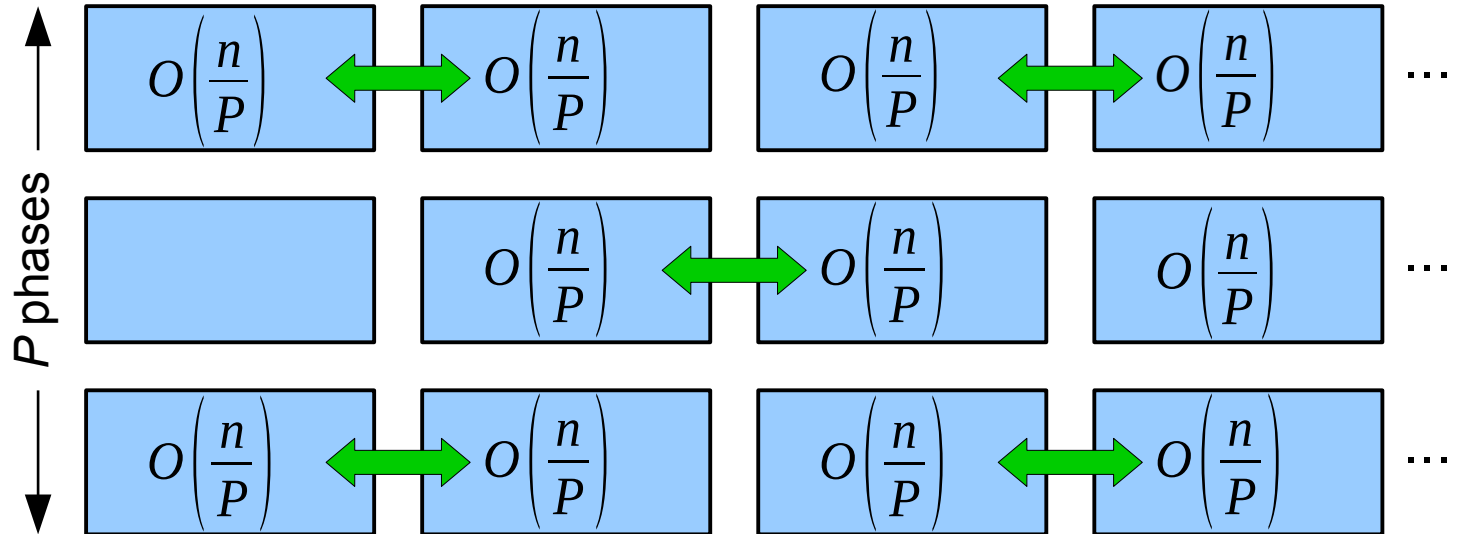
QuickSort

Work = $n \log(n/P)$
Span = $n/P \log(n/P)$



OddEven

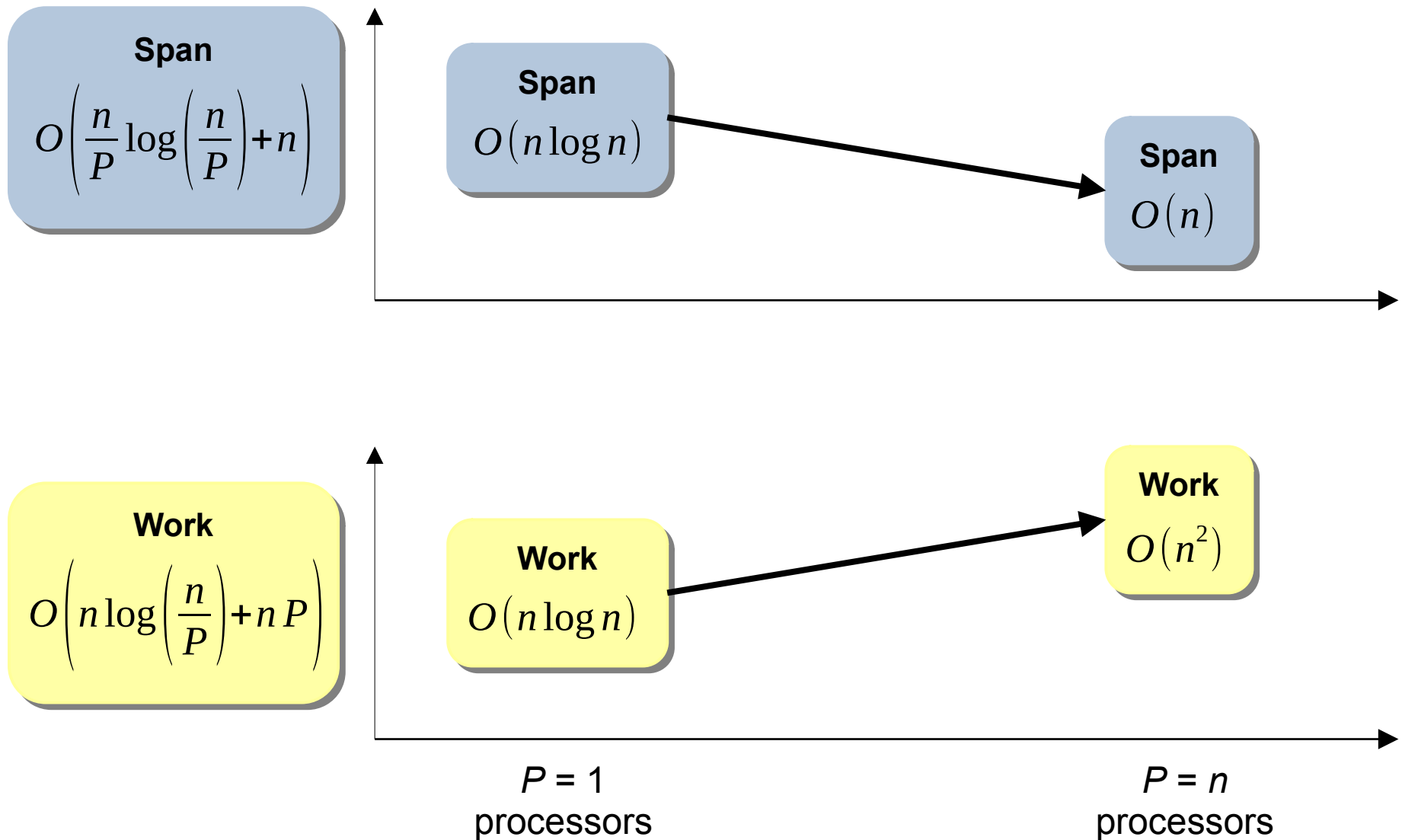
Work = nP
Span = n



- Total Work: $O(n \log (n/P) + nP)$
 - Optimal if $P \ll \log n$
- Total Span: $O(n/P \log (n/P) + n)$

The work also includes exchanging $O(n/P)$ elements with neighbor in $O(n/P)$ time.

Odd-Even Transposition Sort: Analysis



Odd-Even Transposition Sort: Analysis

- A parallel algorithm is said to be **work-optimal** if the work is the same of the best serial algorithm for the same problem.
- **Q:** What is the maximum number of processors P such that the algorithm is work-optimal, that is, the work is proportional to $(n \log n)$?
 - We need to solve $n \log (n / P) + n P = n \log n$ for P

Odd-Even Transposition Sort: Analysis

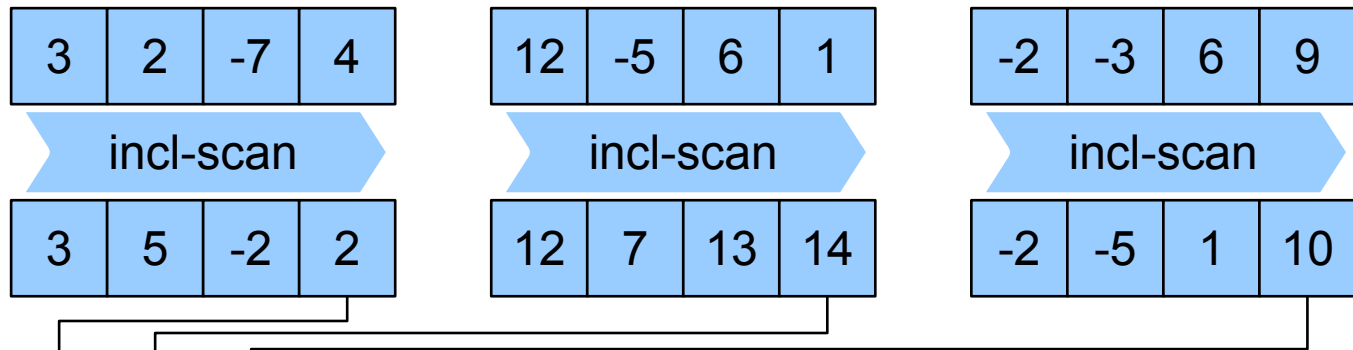
- Solve for P : $n \log (n / P) + n P = n \log n$
- Let us try $P = \log n$
 - $n \log (n / \log n) + n \log n =? n \log n$
 - $n \log n - n \log (\log n) + n \log n =? n \log n$ Yes
- Work
 - We can increase the number of processors up to $\log n$ and still get optimal work
- Span
 - Using $P = \log n$ processors the Span will be $O((n / \log n) (\log n - \log \log n) + n) \sim O(n)$

Domanda 2: Scan inclusiva shared-memory

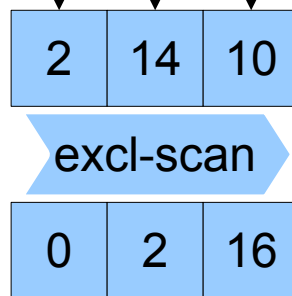
- Calcolare work e span della seguente implementazione della Scan inclusiva considerando una architettura a memoria condivisa.

Domanda 2: Scan inclusiva shared-memory

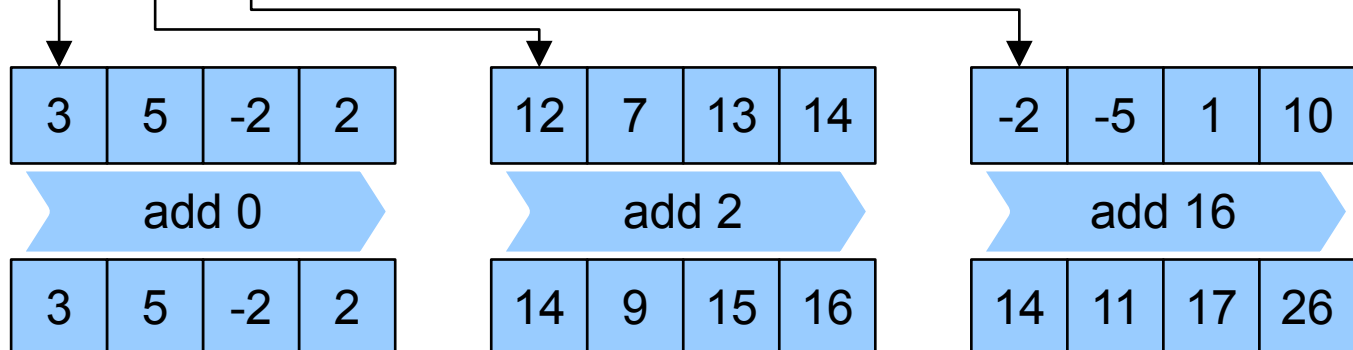
1. Ciascun processo calcola la scan inclusiva di un blocco dell'array di input.



2. Il processo 0 calcola la scan esclusiva dell'ultimo valore di ciascun blocco.



3. Ciascun processo somma alla propria porzione di array il valore corrispondente dell'array del punto precedente.



Domanda 3: Parallel Merge Sort

```
mergesort_rec(v[i..j])
{
    if (i >= j)
        return;
    else {
        m = (i+j)/2;
        #pragma omp task shared(v, i, m, tmp)
        mergesort_rec(v[i..m]);
        #pragma omp task shared(v, j, m, tmp)
        mergesort_rec(v[m+1..j]);
        #pragma omp taskwait
        merge(v[i..m], v[m+1..j]);
    }
}
```

```
/* Invocation */
#pragma omp parallel
#pragma omp single
mergesort_rec(v[0..n-1]);
```

Compute the theoretical work and span of the parallel Merge Sort algorithm.

Assume $P = n$ threads (n = length of array being sorted).

Hints:

- *merge*($v1, v2$) requires time proportional to *length*($v1$) + *length*($v2$).
- $n + n/2 + n/4 + \dots$ is $O(n)$

Domanda 4

- Descrivere un algoritmo per architetture a memoria condivisa usando MPI per risolvere il seguente problema:

È dato un array $v[]$ di lunghezza n . Determinare se l'array soddisfa la proprietà $v[i] == i$ per ogni i .

- Si assuma che:
 - n sia un multiplo intero del numero P di processi MPI.
 - Tutti i processi conoscano il valore di n (e ovviamente di P).
 - È sufficiente che il processo master conosca il risultato della computazione.

Domanda 5

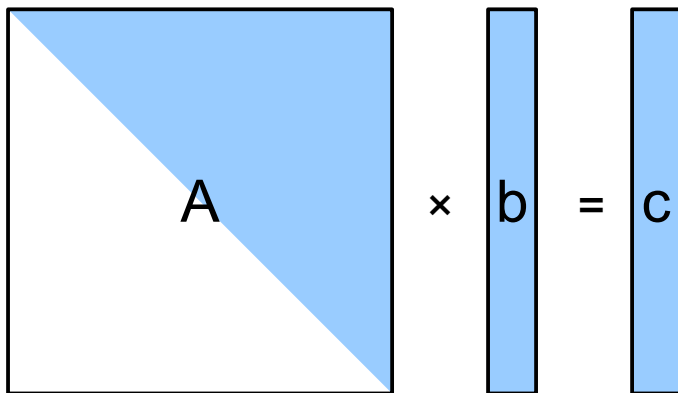
- La funzione seguente effettua una rotazione a destra di un array `a[]` di lunghezza `n`. Si spieghi come parallelizzarla:
 - su architetture a memoria condivisa, e
 - su architetture a memoria distribuita usando

Assumere che `n` sia multiplo del numero di unità di esecuzione.

```
void vec_shift_right(int *a, int n)
{
    const int tmp = a[n-1];
    for (int i=n-1; i>0; i--) {
        a[i] = a[i-1];
    }
    a[0] = tmp;
}
```


Domanda 6

- Sia $A[n][n]$ una matrice in forma triangolare superiore, e $b[n]$ un array; considerando una architettura a memoria condivisa, discutere la parallelizzazione mediante OpenMP del seguente frammento di codice che calcola il prodotto matrice-vettore $Ab = c$.



```
for (int i=0; i<n; i++) {  
    c[i] = 0;  
    for (int j=i; j<n; j++) {  
        c[i] += A[i][j] * b[j];  
    }  
}
```

